<h1 style='color:blue' align='center'>Evaluation Metrics for Classification Models</h1>
Lab Objective: Understand and compare classification evaluation metrics using Logistic Regression, K-Nearest Neighbour (KNN), and Decision Tree models. In this lab, you will calculate and interpret:

- Accuracy
- Precision
- Recall
- F1-score
- ROC Curve and AUC
- Precision-Recall Curve

Dataset: Breast Cancer Wisconsin dataset available through `scikit-learn`.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    roc_auc_score,
    precision_recall_curve,
    average_precision_score
)
```

<h2 id="2.-Load-and-Explore-the-Dataset">2. Load and Explore the Dataset</h2>
The dataset is a binary classification problem. Each sample has numerical features and a binary target label.

In [2]:
```
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name='target')

print('Number of samples:', X.shape[0])
print('Number of features:', X.shape[1])
print('Target classes:', data.target_names)

X.head()
```

```
Number of samples: 569
Number of features: 30
Target classes: ['malignant' 'benign']
```

Out[2]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>mean radius</th> <th>mean texture</th> <th>mean perimeter</th> <th>mean area</th> <th>mean smoothness</th> <th>mean compactness</th> <th>mean concavity</th> <th>mean concave points</th> <th>mean symmetry</th> <th>mean fractal dimension</th> <th>...</th> <th>worst radius</th> <th>worst texture</th> <th>worst perimeter</th> <th>worst area</th> <th>worst smoothness</th> <th>worst compactness</th> <th>worst concavity</th> <th>worst concave points</th> <th>worst symmetry</th> <th>worst fractal dimension</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678

5 rows × 30 columns

In [3]:
```
print('Class distribution:')
print(y.value_counts().sort_index())

print('\nClass proportions:')
print(y.value_counts(normalize=True).sort_index())
```

```
Class distribution:
target
0    212
1    357
Name: count, dtype: int64

Class proportions:
target
0    0.372583
1    0.627417
Name: proportion, dtype: float64
```

<h2 id="3.-Create-Training-and-Test-Sets">3. Create Training and Test Sets</h2>
Use an 80:20 train-test split. <code>stratify=y</code> preserves the class proportions in both sets.

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random

print('Training samples:', X_train.shape[0])
print('Testing samples :', X_test.shape[0])
```

```
Training samples: 455
Testing samples : 114
```

<h2 id="4.-Train-Three-Classification-Models">4. Train Three Classification Models</h2>
We use:

1. Logistic Regression
2. K-Nearest Neighbour (KNN)
3. Decision Tree

Note: Logistic Regression and KNN are trained with feature scaling. Decision Trees do not require feature scaling.

In [5]:
```
from sklearn.preprocessing import StandardScaler

# Feature scaling for Logistic Regression and KNN
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Create the models
log_reg = LogisticRegression(max_iter=1000, random_state=42)
knn = KNeighborsClassifier(n_neighbors=5)
dt = DecisionTreeClassifier(random_state=42)

# Train the models
log_reg.fit(X_train_scaled, y_train)
knn.fit(X_train_scaled, y_train)
dt.fit(X_train, y_train)

print('All three models trained successfully')
```

```
All three models trained successfully
```

<h2 id="5.-Generate-Predictions">5. Generate Predictions</h2> For each model, generate predictions for the unseen test set.

In [6]:
```
predictions = {}

predictions['Logistic Regression'] = log_reg.predict(X_test_scaled)
predictions['KNN'] = knn.predict(X_test_scaled)
predictions['Decision Tree'] = dt.predict(X_test)

for name, y_pred in predictions.items():
    print(name)
    print(y_pred[:10])
    print()
```

```
Logistic Regression
[0 1 0 1 0 1 1 0 0 0]

KNN
[0 1 0 0 0 1 1 0 0 0]

Decision Tree
[0 1 0 1 0 1 1 0 0 0]
```

<h2 id="6.-Evaluation-Metrics:-Accuracy,-Precision,-Recall-and-F1-score">6. Evaluation Metrics: Accuracy, Precision, Recall and F1-score</h2> For a binary classification

problem:

- **Accuracy:** Overall proportion of correct predictions.
- **Precision:** Of the samples predicted as positive, how many are actually positive?
- **Recall:** Of the actual positive samples, how many are correctly identified?
- **F1-score:** Harmonic mean of precision and recall.

Use the test set to calculate all four metrics.

In [7]:
```
results = []

for name, y_pred in predictions.items():
    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1-score': f1_score(y_test, y_pred)
    })

results_df = pd.DataFrame(results)
results_df.round(4)
```

Out[7]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Model</th> <th>Accuracy</th> <th>Precision</th> <th>Recall</th> <th>F1-score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

Logistic Regression	0.9825	0.9861	0.9861	0.9861
KNN	0.9561	0.9589	0.9722	0.9655
Decision Tree	0.9123	0.9559	0.9028	0.9286

<h3 id="Think-and-Discuss">Think and Discuss</h3>

1. Which metric is highest/lowest for each model?
2. Is accuracy alone sufficient to compare the three models?
3. What does a difference between precision and recall tell us?
4. Which metric would you emphasize if missing a positive case is particularly costly?

<h2 id="7.-Confusion-Matrix">7. Confusion Matrix</h2> A confusion matrix shows the four types of outcomes:

- **TP:** True Positive
- **TN:** True Negative
- **FP:** False Positive
- **FN:** False Negative

Generate the confusion matrix for each model.

In [8]:
```
for name, y_pred in predictions.items():
    print('\n' + '=' * 50)
    print(name)
```

```
    print('=' * 50)
    print(confusion_matrix(y_test, y_pred))
```

```
==================================================
Logistic Regression
==================================================
[[41  1]
 [ 1 71]]

==================================================
KNN
==================================================
[[39  3]
 [ 2 70]]

==================================================
Decision Tree
==================================================
[[39  3]
 [ 7 65]]
```

<h2 id="8.-Classification-Report">8. Classification Report</h2> The classification report provides precision, recall and F1-score for each class.

In [16]:
```
for name, y_pred in predictions.items():
    print('\n' + '=' * 60)
    print(name)
    print('=' * 60)
    print(classification_report(y_test, y_pred, target_names=data.target_names))
```

```
============================================================
Logistic Regression
============================================================
              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114


============================================================
KNN
============================================================
              precision    recall  f1-score   support

   malignant       0.95      0.93      0.94        42
      benign       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114


============================================================
Decision Tree
============================================================
              precision    recall  f1-score   support

   malignant       0.85      0.93      0.89        42
      benign       0.96      0.90      0.93        72

    accuracy                           0.91       114
   macro avg       0.90      0.92      0.91       114
weighted avg       0.92      0.91      0.91       114
```

<h2 id="9.-ROC-Curve-and-AUC-for-Logistic-Regression">9. ROC Curve and AUC for Logistic Regression</h2> Unlike accuracy, precision, recall and F1-score at one fixed threshold, the **ROC curve** evaluates the model over different classification thresholds. The ROC curve plots: For Logistic Regression, obtain the probability of the positive class using `predict_proba()`.

In [17]:
```
y_prob_lr = log_reg.predict_proba(X_test_scaled)[:, 1]

fpr, tpr, thresholds = roc_curve(y_test, y_prob_lr)
auc_lr = roc_auc_score(y_test, y_prob_lr)

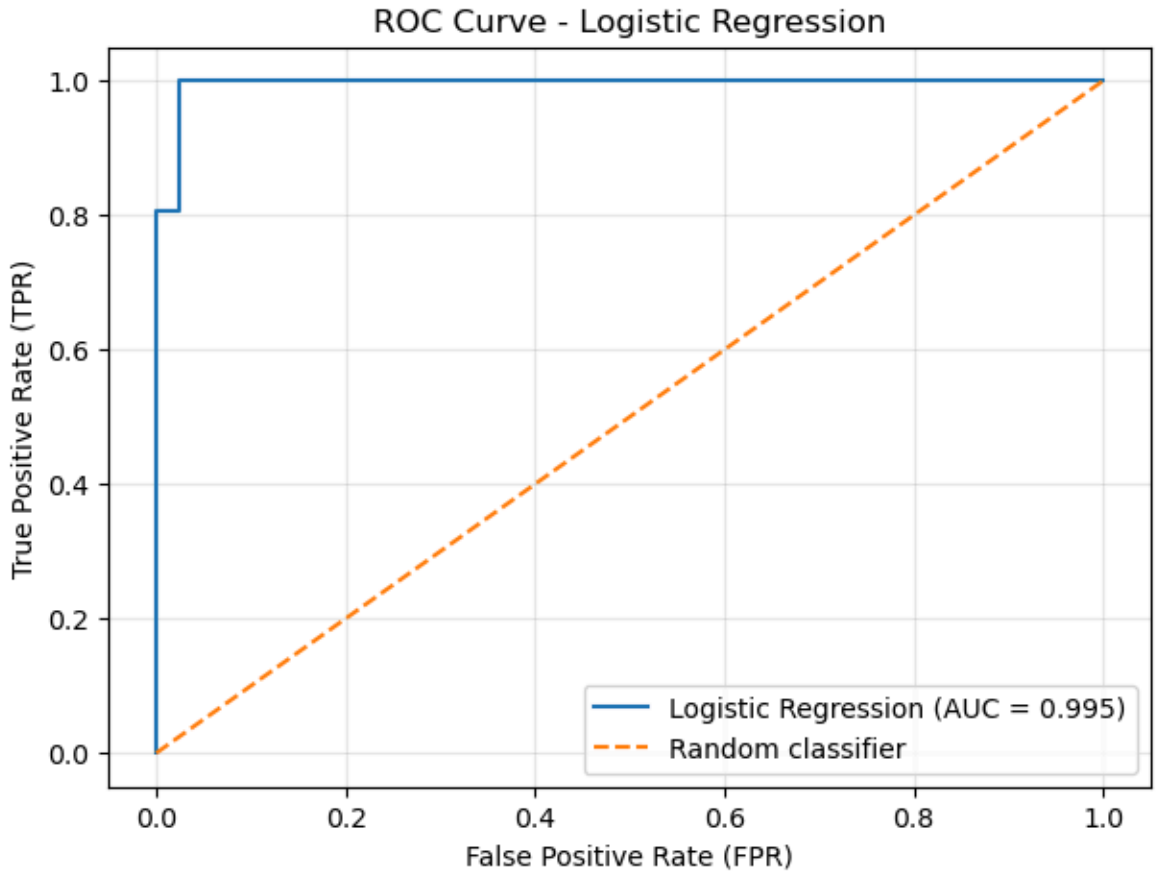
print('AUC =', round(auc_lr, 4))
thresholds
```

```
AUC = 0.9954
```

Out[17]:
```
array([           inf, 9.99999580e-01, 9.16588147e-01, 9.08728148e-01,
       3.65861856e-01, 1.63299654e-23])
```

In [11]:
```
plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, label=f'Logistic Regression (AUC = {auc_lr:.3f})')
```

```
plt.plot([0, 1], [0, 1], linestyle='--', label='Random classifier')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC Curve - Logistic Regression')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



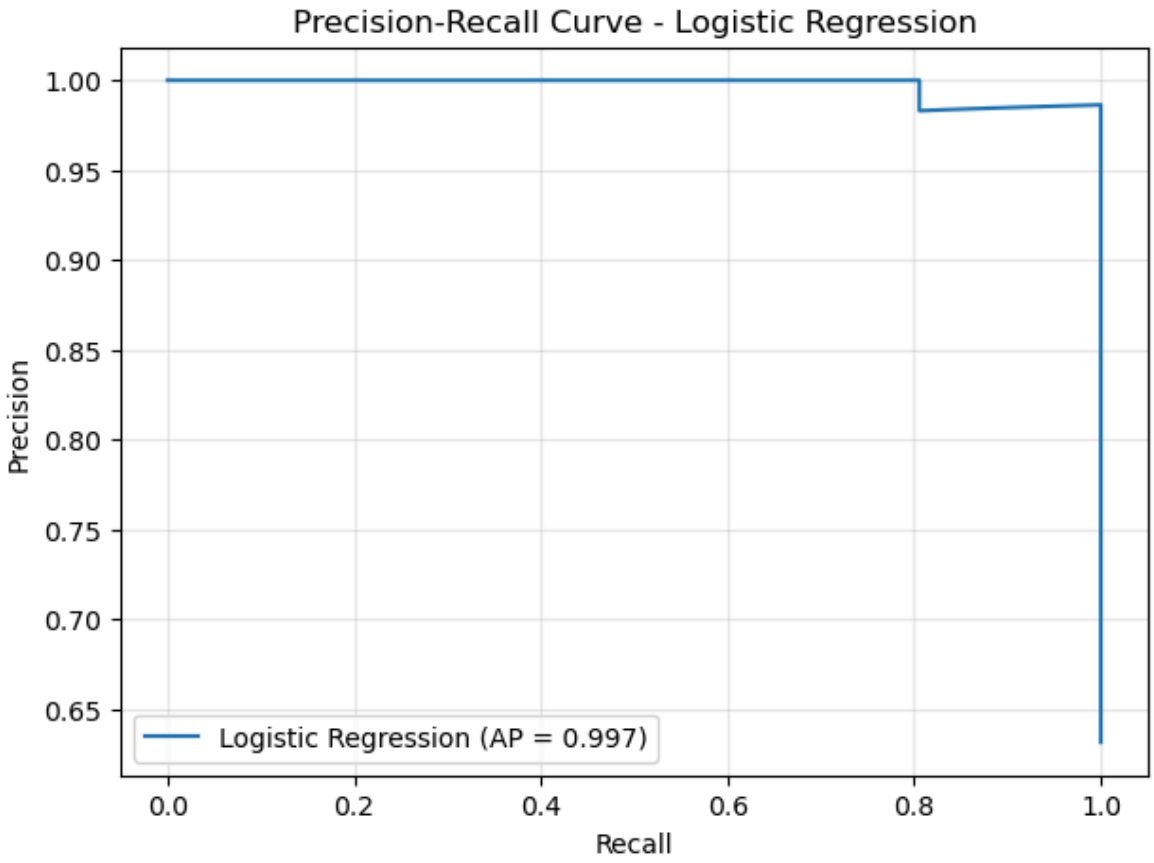
<h2 id="10.-Precision-Recall-Curve-for-Logistic-Regression">10. Precision-Recall Curve for Logistic Regression</h2> The Precision-Recall (PR) curve shows the relationship between: It is particularly useful when the positive class is relatively rare or when false positives and false negatives are important.

In [12]:
```
precision, recall, pr_thresholds = precision_recall_curve(y_test, y_prob_lr)
ap_lr = average_precision_score(y_test, y_prob_lr)

print('Average Precision =', round(ap_lr, 4))
```

```
Average Precision = 0.9971
```

In [13]:
```
plt.figure(figsize=(7, 5))
plt.plot(recall, precision, label=f'Logistic Regression (AP = {ap_lr:.3f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve - Logistic Regression')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



<h3 id="ROC-vs-Precision-Recall-Curve">ROC vs Precision-Recall Curve</h3> <thead> <th>ROC Curve</th> <th>Precision-Recall Curve</th> </thead> <tbody> </tbody> <tbody></tbody>

TPR vs FPR	Precision vs Recall
Shows sensitivity to false positives through FPR	Directly shows the precision-recall trade-off
AUC-ROC summarizes ranking performance	Average Precision summarizes the PR curve

Discuss: Why might a Precision-Recall curve be more informative than an ROC curve for a highly imbalanced classification problem?

<h2 id="11.-Threshold-Analysis-for-Logistic-Regression">11. Threshold Analysis for Logistic Regression</h2> The default classification threshold is usually 0.5. In this activity, investigate how changing the threshold affects accuracy, precision, recall and F1-score.

In [14]:
```
thresholds_to_test = [0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]

threshold_results = []

for t in thresholds_to_test:
    y_pred_t = (y_prob_lr >= t).astype(int)
    threshold_results.append({
        'Threshold': t,
        'Accuracy': accuracy_score(y_test, y_pred_t),
        'Precision': precision_score(y_test, y_pred_t, zero_division=0),
        'Recall': recall_score(y_test, y_pred_t, zero_division=0),
        'F1-score': f1_score(y_test, y_pred_t, zero_division=0)
    })
```

```
pd.DataFrame(threshold_results).round(4)
```

Out[14]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Threshold</th> <th>Accuracy</th> <th>Precision</th> <th>Recall</th> <th>F1-score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody> </tbody>

0.2	0.9825	0.9730	1.0000	0.9863
0.3	0.9825	0.9730	1.0000	0.9863
0.4	0.9825	0.9861	0.9861	0.9861
0.5	0.9825	0.9861	0.9861	0.9861
0.6	0.9561	0.9855	0.9444	0.9645
0.7	0.9474	0.9853	0.9306	0.9571
0.8	0.9386	0.9851	0.9167	0.9496

<h3 id="Interpretation-Questions">Interpretation Questions</h3>

1. As the threshold increases, what generally happens to recall?
2. What generally happens to precision?
3. At which threshold is the F1-score highest for this test set?
4. Why can changing the threshold change precision and recall without retraining the model?

<h2 id="12.-Lab-Exercise">12. Lab Exercise</h2> **Complete the following tasks independently on IRIS dataset:**

1. Change the KNN value of `n_neighbors` to 3, 7 and 11. Compare accuracy, precision, recall and F1-score.
2. Change the Decision Tree `max_depth` to 2, 4 and 6. Compare the four evaluation metrics.
3. Plot the Precision-Recall curves for all three models.
4. Identify a threshold for Logistic Regression that provides a useful balance between precision and recall, and justify your choice using the results.

<h2 id="13.-Submission-/-Reflection">13. Submission / Reflection</h2> Prepare a short table containing the following for all three models: <thead> <th>Model</th> <th style="text-align:right">Accuracy</th> <th style="text-align:right">Precision</th> <th style="text-align:right">Recall</th> <th style="text-align:right">F1-score</th> <th style="text-align:right">ROC-AUC</th> </thead> <tbody> </tbody> <tbody></tbody>

Logistic Regression	
KNN	

Decision Tree

Finally, write **3--5 observations** explaining how the evaluation metrics differ across the models and how the classification threshold affects Logistic Regression.